

CMSE 801 - Day 4



Practice with `if` statements and functions;
Revisiting the savings model

September 9, 2024

General Course Announcements

- Homework 1 should come out on Wednesday of this week. This assignment will be **due at 11:59pm on Friday, September 27.**
 - This will be an opportunity to showcase all of the Python fundamentals you've picked up in course up to this point.
- Reminder: office hours/help room details are on the course homepage:
<https://cmse.msu.edu/cmse801>
- Remember: at the end of class **you will be uploading your in-class notebook to D2L.**

Questions/comments from Day 4 PCA

- Work with "if statements" largely seemed to go OK for folks
- But also: writing more complex code and including conditional statements can lead to more bugs/errors, making reading the error outputs more important -- **carefully review your code when you run into errors!**
- General questions around writing functions from scratch and how they work
 - Some specific questions about the hypotenus example -- **we'll look at that one**
- Some questions about the `return` statement, how to use it, and if more than one thing can be returned
- Some questions and confusion about "variable scope"
 - Python is pretty lax about variable scope and jupyter notebooks can make this a bit more confusing since you can run code "out of order". This can make things even more confusing!
 - **We'll explore this a little bit as well**

The importance of knowing if your code is "inside" or "outside" your conditional statement

```
In [1]: # initializing the list
greater_than_6_list = []
for i in range(11):
    print("i equals",i)
    if i > 6:
        print("i is greater than 6")
        # appending the number that is greater than 6
        greater_than_6_list.append(i) # I need to make sure this is "inside"
print("List of numbers to sum:",greater_than_6_list)
print("Sum of numbers is equal to:",sum(greater_than_6_list))
```

```
i equals 0
i equals 1
i equals 2
i equals 3
i equals 4
i equals 5
i equals 6
i equals 7
i is greater than 6
i equals 8
i is greater than 6
i equals 9
i is greater than 6
i equals 10
i is greater than 6
List of numbers to sum: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Sum of numbers is equal to: 55
```

Demystifying function creation

What are the important components needed when creating a function?

Let's list some important parts of writing functions:

- `def` -- defines the function
- name (comes right after `def`)
- arguments -- inside `()`
- `return` -- use this to get something back from the function
- It should serve a purpose

```
In [2]: # Calculating the hypotenuse of a triangle  
a = 3  
b = 4  
hyp = (a**2 + b**2)**(1/2)  
print(hyp)
```

5.0

```
In [3]: # Can we create a function that does this?
```

```
In [5]: # Let's try running it here for 5 and 12  
result = get_h(5,12)  
print(result)
```

13.0

What if we wanted to return more than one thing?

```
In [6]: def get_h(a,b):  
        h = (a**2 + b**2)**(1/2)  
        h_times_2 = h * 2.0  
        return h, h_times_2
```

```
In [7]: a = 3  
        b = 4  
        result, result_times_2 = get_h(a,b)  
        print(result, result_times_2)
```

```
5.0 10.0
```


Variable scope (or "all the ways Python will try to drive me crazy")

First, let's look at the recommended approach

```
In [8]: def get_h(a,b):  
        h = (a**2 + b**2)**(1/2)  
        return h
```

```
In [9]: # Define my parameters/variables  
my_a = 3  
my_b = 4  
  
#Call my function  
result = get_h(my_a,my_b)  
print(result)  
  
# Will the code below work? If I haven't defined "h" anywhere outside my  
# If "h" is only defined inside the loop, the rest of my code doesn't ki  
  
# print(h)
```

Thought exercise: will this code work?

```
In [10]: a = 3
b = 4
def get_h():
    h = (a**2 + b**2)**(1/2)
    return h
result = get_h()
print(result)
```

5.0

Disclaimer: don't get in the habit of writing code like this!!

```
In [11]: a = 3
          b = 4
          def get_h():
              h = (a**2 + b**2)**(1/2)
              return h
          result = get_h()
          print(result)
```

5.0

My code is no longer easily reusable later on!

```
In [12]: # Define new parameters/variables
          new_a = 5
          new_b = 12

          #Call my function
          result = get_h(new_a, new_b)
          print(result)
```


TypeError

Traceback (most recent

call last)

Cell In[12], line 6

3 new_b = 12

5 #Call my function

----> 6 result = get_h(new_a, new_b)

7 print(result)

TypeError: get_h() takes 0 positional arguments but 2 were given

Also, when you use default argument values in your function, you can further confuse yourself if you're not being careful!

You'll see more about setting default arguments in the next PCA.

```
In [13]: a = 3
          b = 4
          def get_h(a=5, b=12):
              h = (a**2 + b**2)**(1/2)
              return h
          result = get_h()
          print(result)
```

13.0

Goals for today

- Practice reading and writing `if` statements and more complex options that use `elif` and `else`
- Revisit the savings problem to calculate saving for many different people (*using `if` statements*)
- Re-write your code *using a function*.

Looking forward - Day 5 PCA: Getting more comfort and practice with functions

- Spend more time thinking about and learning about how to write functions
- Spend more time practicing writing functions and using functions

